Weather Forecasting Using Machine Learning

Student Name: [Your Name]

USN: [Your USN]

Department: [Your Department]

College: [Your College]

Guide: [Guide Name]

Academic Year: [202X-202X]

Introduction

- **Weather Forecasting:** Estimating future atmospheric conditions using earlier observations.
- **Role of Machine Learning:** Learning temperature patterns from historical state-level weather data.
- **Project Purpose:** Allowing users to choose a state and date to return a next-day average temperature forecast.
- **Outputs:** Forecasts, Weather History, Model Performance, and Prediction History views.

Problem Statement

To turn stored state-level weather observations into a next-day average-temperature forecast that users can request through a web application.

The project must:

- Use the same features in training and serving.
- Avoid future-data leakage.
- Retain the predictions generated through the API with a complete audit trail.

Objectives

1. **Train and compare regression models**, selecting a model by test RMSE.
2. **Serve predictions** with FastAPI.
3. **Use PostgreSQL** for weather data and audit records.
4. **Deliver four UI views** in React for forecasts, weather history, model performance, and prediction history.

Existing System vs Proposed System

Feature	Manual / Legacy Approach	Proposed System
Forecast Creation	Manual or disconnected modeling	Automated via unified API
Feature Preparation	Manual extraction	Real-time automated feature service
Data Access	Scattered files/scripts	Centralized PostgreSQL database
Validation	Minimal or ad-hoc	Strict FastAPI request validation
Traceability	None	Full audit trail in `predictions` table
		React/Vite interactive

System Architecture

1. **User Interface:** User interacts with the React and Vite frontend.
2. **Backend Engine:** FastAPI validates the request.
3. **Database:** Reads weather history from PostgreSQL.
4. **Feature Service:** Constructs the exact 13 inputs used in training.
5. **Prediction Model:** The saved Gradient Boosting pipeline predicts the temperature.
6. **Audit & Response:** FastAPI stores the audit record and returns the response to the UI.

Technologies Used

- **Frontend:** React, TypeScript, Vite, Recharts
- **Backend:** Python, FastAPI, Uvicorn, SQLAlchemy, psycopg
- **Database:** PostgreSQL
- **Machine Learning:** Pandas, NumPy, scikit-learn, joblib
- **Data Source:** Open-Meteo Archive (for optional manual data-update scripts)

Dataset

- **Source:** Extracted from historical weather records.
- **Size:** 31,177 rows across 34 state labels.
- **Date Range:** 1 April 2023 to 6 October 2025.
- **Variables Selected:** Focuses strictly on weather fields (Max/Min/Avg Temp, Humidity, Rainfall, etc.), excluding power-demand fields.
- **Data Quality:** Zero nulls in the required modeling fields, and zero duplicate state-date pairs.

Machine Learning Model

- **Algorithm:** scikit-learn Gradient Boosting Regressor.
- **Target Variable:** Next-day `Temp_Avg` (Degrees Celsius).
- **Training Methodology:** Chronological split to prevent future-data leakage.
- **Evaluation:** Compared with Linear Regression, Decision Tree, and Random Forest.
- **Results:** Gradient Boosting achieved the lowest error metrics:
  - **MAE:** 0.8760
  - **RMSE:** 1.1854
  - **R-squared:** 0.9694

Feature Engineering

The backend reconstructs 13 inputs used at training time to prevent data leakage:

- 1 to 3 day average temperature lags.
- Previous-day maximum, minimum, humidity, and rainfall.
- 3-day and 7-day rolling temperature averages.
- Cyclic month values (sine and cosine).
- Season and state categories.
  (Target day's actual weather is never used in the forecast)

Backend and API

- **Key Endpoint:** `POST /predict`
- **Validation:** Checks state availability and sufficient historical records.
- **Process Flow:**
  - Validates requested state and date.
  - Fetches last 7 days of history (preventing stale history usage).
  - Builds the feature array.
  - Loads the joblib ML model to make the prediction.
  - Persists the result to the database.
  - Returns structured JSON to the client.

Frontend

The user interface provides four main views:

1. **Forecast:** Creates a prediction and shows a recent history chart.
2. **Weather History:** Displays historical state records with pagination.
3. **Model Performance:** Exposes the metadata and model comparison metrics.
4. **Prediction History:** Lists stored API predictions with a client-side state filter.

Includes shared health, error, and data-fetch behavior.

Database

- `weather_data` **table:** Stores one daily weather record per state. Enforces a unique state-date constraint.
- `predictions` **table:** Stores each generated forecast alongside its creation timestamp.
- **Design Philosophy:** Data supplies forecasting inputs, and predictions act as an output audit trail.

Testing and Results

- **PASS:** Saved model loading, source data checks, and frontend build.
- **WARNING:** JavaScript chunk size warning during UI build (performance notice, not an error).
- **FAIL (Pending):** Live API and Database-dependent tests require a correctly configured and available PostgreSQL instance to complete successfully.

Deployment Architecture

- **Development Setup:** Confirmed local development configuration.
  - Vite on port 5173
  - FastAPI on port 8000
  - PostgreSQL via local connection string
- **CORS:** Environment-controlled allow-list.
- *(Note: Cloud deployment configurations such as Vercel, Render, or Neon are currently unverified.)*

Conclusion and Future Scope

Conclusion:

Implemented a complete local full-stack workflow for next-day average-temperature forecasting across 34 state labels, featuring chronological training, strict API validation, and full UI views.

Future Scope:

- Evaluate deep learning approaches.
- Integrate additional weather attributes (e.g., wind speed).
- Expand regional coverage.
- Automate continuous data ingestion upon cloud deployment.